CO5_AT3_CODE_CHALLENGING

View Only

← Back

QUESTIONS

- Q1 Multimedia Player Abstraction 20 marks
- Q2 Virtual Destructor Debugging 20 marks
- Q3 Static Object Counter 20 marks
- Q4 Exception Handling: Division 20 marks
- Q5 Deep Copy of Student Record 20 marks

5/5 answered

## Q1 Multimedia Player Abstraction

20 marks

Create an abstract MediaPlayer class with a pure virtual play(). Implement MP3Player, WAVPlayer, MP4Player, and AVIPlayer. Select the correct object from the input and invoke play() polymorphically.

### Your Code

```cpp
#include <iostream>
using namespace std;

class MediaPlayer {
public:
    virtual void play() = 0;
    virtual ~MediaPlayer() {}
};

class MP3Player : public MediaPlayer {
public:
    void play() {
        cout << "Playing MP3" << endl;
    }
};

class WAVPlayer : public MediaPlayer {
public:
    void play() {
        cout << "Playing WAV" << endl;
    }
};

class MP4Player : public MediaPlayer {
public:
    void play() {
        cout << "Playing MP4" << endl;
    }
};

class AVIPlayer : public MediaPlayer {
public:
    void play() {
        cout << "Playing AVI" << endl;
    }
};

int main() {
    int choice;
    cin >> choice;

    MediaPlayer* player;

    if (choice == 1)
        player = new MP3Player();
    else if (choice == 2)
        player = new WAVPlayer();
    else if (choice == 3)
        player = new MP4Player();
    else
        player = new AVIPlayer();

    player->play();

    delete player;

    return 0;
}
```

### Input

```
1
```

Q3
Static Object Counter
20 marks

Q4
Exception Handling: Division
20 marks

Q5
Deep Copy of Student Record
20 marks

5/5 answered

```cpp
#include <iostream>
using namespace std;

class MediaPlayer {
public:
    virtual void play() = 0;
    virtual ~MediaPlayer() {}
};

class MP3Player : public MediaPlayer {
public:
    void play() {
        cout << "Playing MP3" << endl;
    }
};

class WAVPlayer : public MediaPlayer {
public:
    void play() {
        cout << "Playing WAV" << endl;
    }
};

class MP4Player : public MediaPlayer {
public:
    void play() {
        cout << "Playing MP4" << endl;
    }
};

class AVIPlayer : public MediaPlayer {
public:
    void play() {
        cout << "Playing AVI" << endl;
    }
};

int main() {
    int choice;
    cin >> choice;

    MediaPlayer* player;

    if (choice == 1)
        player = new MP3Player();
    else if (choice == 2)
        player = new WAVPlayer();
    else if (choice == 3)
        player = new MP4Player();
    else
        player = new AVIPlayer();

    player->play();

    delete player;

    return 0;
}
```

Input

```
1
```

Output

Visible Test Cases

Test Case 1

# CO5_AT3_CODE_CHALLENGING

View Only

← Back

QUESTIONS

- Q1 Multimedia Player Abstraction 20 marks
- Q2 Virtual Destructor Debugging 20 marks
- Q3 Static Object Counter 20 marks
- Q4 Exception Handling: Division 20 marks
- Q5 Deep Copy of Student Record 20 marks

5/5 answered

## Q2 Virtual Destructor Debugging

20 marks

Debug the following inheritance program. A derived object is deleted through a base pointer. Modify the base class so that both destructors execute safely and in the correct order.

```
class Parent {
public:
~Parent() { cout << "Parent Destructor" << endl; }
};
class Child : public Parent {
public:
~Child() { cout << "Child Destructor" << endl; }
};
int main() {
Parent* p = new Child();
delete p;
}
```

### Your Code

```cpp
#include <iostream>
using namespace std;

class Parent {
public:
    virtual ~Parent() {
        cout << "Parent Destructor" << endl;
    }
};

class Child : public Parent {
public:
    ~Child() {
        cout << "Child Destructor" << endl;
    }
};

int main() {
    Parent* p = new Child();

    delete p;

    return 0;
}
```

### Input

stdin input

### Output

### Visible Test Cases

Test Case 1

QUESTIONS

- Q1 Multimedia Player Abstraction 20 marks
- Q2 Virtual Destructor Debugging 20 marks
- Q3 Static Object Counter 20 marks
- Q4 Exception Handling: Division 20 marks
- Q5 Deep Copy of Student Record 20 marks

5/5 answered

## Q3 Static Object Counter

20 marks

Create a static data member in Employee to count all Employee objects created. The count must be shared across objects and accessible without maintaining a separate counter in each object.

### Your Code

```cpp
#include <iostream>
using namespace std;

class Employee {
    static int count;

public:
    Employee() {
        count++;
    }

    static void showCount() {
        cout << count << endl;
    }
};

int Employee::count = 0;

int main() {
    int n;
    cin >> n;

    Employee emp[n];

    Employee::showCount();

    return 0;
}
```

### Input

```
5
```

### Output

### Visible Test Cases

Test Case 1

QUESTIONS

- Q1 Multimedia Player Abstraction 20 marks
- Q2 Virtual Destructor Debugging 20 marks
- Q3 Static Object Counter 20 marks
- Q4 Exception Handling: Division 20 marks
- Q5 Deep Copy of Student Record 20 marks

5/5 answered

## Q4 Exception Handling: Division

20 marks

Create a Calculator class with a divide() method that throws an exception when you attempt division by zero. Catch the exception in main() and display an informative message.

```
class Calculator {
public:
double divide(double a, double b) {
return a / b;
}
};
```

### Your Code

```cpp
#include <iostream>
using namespace std;

class Calculator {
public:
    double divide(double a, double b) {
        if (b == 0)
            throw "Division by zero is not allowed";

        return a / b;
    }
};

int main() {
    double a, b;
    cin >> a >> b;

    Calculator c;

    try {
        cout << c.divide(a, b) << endl;
    }
    catch (const char* msg) {
        cout << msg << endl;
    }

    return 0;
}
```

### Input

```
10 2
```

### Output

### Visible Test Cases

Test Case 1

QUESTIONS



5/5 answered

## Q5 Deep Copy of Student Record

20 marks

Debug a class containing dynamically allocated marks. Implement a deep-copy constructor and destructor. Changing one object's marks must not change the copied object.

```
class Student {
int* marks;
public:
Student(int m) { marks = new int(m); }
Student(const Student& s) {
marks = s.marks; // Bug: shallow copy
}
void setMarks(int m) { *marks = m; }
void display() { cout << *marks << endl; }
};
```

### Your Code

```cpp
#include <iostream>
using namespace std;

class Student {
    int* marks;

public:
    Student(int m) {
        marks = new int(m);
    }

    // Deep copy constructor
    Student(const Student& s) {
        marks = new int(*s.marks);
    }

    // Destructor
    ~Student() {
        delete marks;
    }

    void setMarks(int m) {
        *marks = m;
    }

    void display() {
        cout << *marks << endl;
    }
};

int main() {
    int m, newMarks;

    cin >> m;
    cin >> newMarks;

    Student s1(m);
    Student s2 = s1;

    s2.setMarks(newMarks);

    s1.display();
    s2.display();

    return 0;
}
```

### Input

```
80
95
```

### Output

Visible Test Cases